

ANALISE COMPLETA DO SISTEMA EVYRA

evyra.site — Plataforma de Saude Domiciliaria

Estudo arquitetural, funcional, de seguranca e de melhoria

Repositorio: github.com/isanat/careapp

Data: 14 de Junho de 2026
Gerado por Z.ai

Sumario

1. Visao Geral da Plataforma
 2. Arquitetura e Stack Tecnologico
 3. Modelos de Dados (35 Modelos)
 4. Funcionalidades Implementadas
 5. Sistema de Autenticacao e Autorizacao
 6. Arquitetura de Pagamentos
 7. Sistema KYC (Verificacao de Identidade)
 8. Painel Administrativo
 9. Estado do Site em Producao
 10. Analise de Seguranca — Falhas Criticas
 11. Pontos de Dor e Gargalos
 12. Sugestoes de Melhoria
 13. Roadmap — O Que Falta Implementar
 14. Conclusao e Recomendacoes Prioritarias
-

1. Visao Geral da Plataforma

O **Evyra** (evyra.site) e uma plataforma portuguesa de saude-tecnologia que conecta familias a profissionais de saude verificados que prestam cuidados ao domicilio. O slogan da plataforma e "Saude que vem ate si" e o publico-alvo principal sao membros da diaspورا portuguesa espalhada pela Europa (Franca, Reino Unido, Alemanha, Suica) que tem pais idosos em Portugal e necessitam de coordenar e monitorizar os cuidados de saude a distancia.

A plataforma atua como intermediaria entre familias (que contratam) e profissionais de saude (que prestam servicos), oferecendo verificacao de identidade, contratos digitais, pagamentos seguros em escrow via Stripe, e confirmacao de presenca via QR code. O modelo de receita baseia-se numa taxa de ativacao de 35 EUR para familias, uma taxa de contrato de 5 EUR por contrato, e uma comissao de 10-15% sobre cada pagamento de servico. O registo e gratuito para profissionais de saude.

1.1 Dados Cadastrais

Campo	Valor
Dominio	evyra.site (alternativo: evyra.eu)
Idioma	Portugues (pt_PT)

Campo	Valor
Mercados	Portugal, Espanha, Italia — expansao Europa
Email	contacto@evyra.eu / suporte@evyra.eu
Morada	Av. da Liberdade, 123, 1250-096 Lisboa
Telefone	+351 912 345 678 (possivel placeholder)
Copyright	2026 Evyra

1.2 Estado Atual

A plataforma encontra-se em fase muito inicial de lancamento. A homepage exhibe contadores com valores zero (0+ profissionais, 0 paises, 0% satisfacao), enquanto a pagina "Sobre" contraditoriamente afirma "50+ Profissionais verificados" e "100+ Familias atendidas". Nao existe qualquer presenca nos motores de busca — pesquisas por "evyra.site" ou "evyra saude" retornam zero resultados organicos. A plataforma ainda nao possui usuarios reais e necessita de esforco significativo de go-to-market.

2. Arquitetura e Stack Tecnologico

2.1 Frontend

Tecnologia	Versao	Uso
Next.js	16.1.1	Framework principal (App Router, SSR/SSG)
React	19.0.0	UI library
Tailwind CSS	4.x	Styling utility-first
Radix UI	varios	Componentes acessiveis (20+ pacotes)
Framer Motion	12.23	Animacoes e transicoes
shadcn/ui	custom	Design system (Bloom customizado)
Recharts	2.15	Graficos no painel admin
next-themes	0.4.6	Tema claro/escuro
Zustand	5.0.6	State management

2.2 Backend

Tecnologia	Versao	Uso
Next.js API Routes	16.1.1	API REST (90+ endpoints)
Prisma ORM	6.11.1	ORM + migracoes

Tecnologia	Versao	Uso
PostgreSQL	16	Banco de dados principal
NextAuth.js	4.24.11	Autenticacao JWT
Stripe	20.3.1	Pagamentos + escrow + Connect
Easypay	custom	MB Way, Multibanco, CC (Portugal)
Didit (KYC)	custom	Verificacao de identidade
Socket.IO	4.8.3	Chat em tempo real (micro-servico)
Agora RTC	4.24	Video-chamadas para entrevistas
Jitsi Meet	1.4.4	Video-conferencia alternativa
Nodemailer	8.0.1	Envio de emails

2.3 Infraestrutura

Componente	Detalhes
Servidor	VPS (164.68.126.14), 11GB RAM, 194GB disco, 61% usado
Orquestracao	Coolify (PaaS self-hosted) na porta 8000
Container App	evyra-app (Next.js standalone, porta 3000)
Container Chat	evyra-chat (Bun, porta 3003)
Container DB	evyra-postgres (porta 5437 externa)
Reverse Proxy	Coolify Proxy (portas 80, 443, 8080)
Caddy	Configurado para WebSocket e chat service
Rede Docker	evyra-network (bridge entre containers)
Dockerfile	Multi-stage (deps, build, runner), non-root user
Volumes	evyra-uploads + evyra-uploads-public (persistentes)

O stack e moderno e bem escolhido. A combinacao Next.js 16 + React 19 + Prisma + PostgreSQL e solida e escalavel. A escolha de Coolify como PaaS self-hosted reduz custos operacionais em comparacao com Vercel/AWS, mas exige mais manutencao manual. O micro-servico de chat em Bun separa corretamente responsabilidades de tempo real da aplicacao principal.

3. Modelos de Dados (35 Modelos)

O schema Prisma define 12 enums e 35 modelos ativos (mais 6 comentados para a Fase 2). A base de dados e PostgreSQL com migracoes gerenciadas pelo Prisma. Abaixo os modelos principais organizados por dominio funcional.

3.1 Enums (12)

Enum	Valores	Proposito
UserRole	FAMILY, CAREGIVER, ADMIN	Classificacao primaria do utilizador
AdminRole	SUPER_ADMIN, ADMIN, SUPPORT, MODERATOR, ANALYST	Permissoes granulares de admin
UserStatus	PENDING, ACTIVE, SUSPENDED, INACTIVE	Ciclo de vida da conta
VerificationStatus	UNVERIFIED, PENDING, VERIFIED, REJECTED	Verificacao de identidade
PaymentType	ACTIVATION, CONTRACT_FEE, SERVICE_PAYMENT, VISIBILITY_BOOST	Categorias de pagamento
PaymentStatus	PENDING, PROCESSING, COMPLETED, FAILED, REFUNDED	Ciclo de vida do pagamento
PaymentProvider	STRIPE, EASYPAY, BANK_TRANSFER, INTERNAL	Gateway de pagamento
ContractStatus	DRAFT ate COMPLETED/CANCELLED/DISPUTED (8 estados)	Ciclo de vida do contrato
ServiceType	18 tipos (PERSONAL_CARE a WOUND_CARE)	Taxonomia de servicos
DemandStatus	ACTIVE, CLOSED, PAUSED, EXPIRED	Estados do marketplace
VisibilityPackage	NONE, BASIC, PREMIUM, URGENT	Tiers de promocao de demandas
ExperienceLevel	BEGINNER, INTERMEDIATE, ADVANCED, EXPERT	Qualificacao do cuidador

3.2 Modelos por Dominio

Usuarios e Perfis

Modelo	Campos-Chave	Proposito
User	email, phone, passwordHash, role, status, kycFields	Entidade central do utilizador
ProfileFamily	elderName, elderAge, medicalConditions, emergencyContacts	Perfil do familiar/idoso
ProfileCaregiver	title, bio, hourlyRateEur, certifications, IBAN	Perfil do profissional
AdminUser	adminRole, customPermissions, 2FA fields	Perfis administrativos

Contratos e Pagamentos

Modelo	Campos-Chave	Proposito
Contract	familyUserId, caregiverUserId, hourlyRateEur, scheduleJson, weeklyPaymentEnabled	Acordos de servico
ContractAcceptance	IP, userAgent, timestamps (ambas as partes)	Prova legal de aceite
Payment	type, status, provider, amountEurCents, platformFee	Registos de pagamento
EscrowPayment	totalAmountCents, platformFeeCents, transferId	Pagamentos em escrow Stripe
WeeklyPaymentApproval	weekNumber, weeklyAmountCents, familyDecision	Aprovacao semanal de pagamentos
CaregiverTransfer	grossAmount, platformFeePercent, netAmount, stripeTransferId	Historico de transferencias
Receipt	receiptNumber (RC-YYYY-NNNNNN), NIFs	Recibos fiscais
RecurringPayment	stripeSubscriptionId, billingDay, period	Pagamentos recorrentes

Marketplace e Demandas

Modelo	Campos-Chave	Proposito
Demand	familyUserId, title, serviceTypes, budgetEurCents, visibilityPackage	Demandas de cuidados
DemandView	demandId, caregiverId, viewedAt	Analytics de visualizacoes
VisibilityPurchase	package, amountEurCents, expiresAt	Compras de boost de visibilidade
Proposal	message, proposedHourlyRate, estimatedStartDate, status	Propostas de cuidadores
Interview	scheduledAt, videoRoomUrl, questionnaireJson	Entrevistas por video

Comunicacao e Notificacoes

Modelo	Campos-Chave	Proposito
ChatRoom/ChatMessage	type, referenceType, message types (text/image/file/system)	Sistema de mensagens
Notification	type, title, message, email/push delivery tracking	Alertas ao utilizador

Modelo	Campos-Chave	Proposito
PresenceConfirmation	qrCode (24h validity), scannedByUserId, status	Check-in via QR code
Review	rating (1-5), sub-ratings (4 dimensoes), moderation	Avaliacoes mutuas

Administracao e Compliance

Modelo	Campos-Chave	Proposito
AdminAction	action, entityType, entityId, oldValue/newValue JSON, IP	Trilha de auditoria
SupportTicket	categories, priorities, assignment, resolution tracking	Sistema de suporte
ModerationQueue	entityType, reason, status, reviewedBy	Moderacao de conteudo
TermsAcceptance	termsType, version, IP, userAgent	Prova de aceite legal
ImpersonationLog	adminUserId, targetUserId, reason, duration	Log de impersonacao admin
PlatformSettings	activationCostEurCents(3500), platformFeePercent(15)	Configuracao dinamica
SiteSettings	logoUrl, faviconUrl, heroImageUrl	Branding do site
PlatformMetric	metricKey, metricValue, period	Metricas analiticas

O schema e abrangente e bem estruturado, cobrindo todo o ciclo de vida da plataforma — desde o registro e verificacao ate ao pagamento e conformidade legal. No entanto, os modelos da Fase 2 (CaregiverWallet, CaregiverWithdrawal, FamilyPayment, FamilyCredit, StripePaymentHold) estao comentados, o que significa que partes importantes do fluxo financeiro ainda nao estao totalmente implementadas no ORM. Atualmente, referencias como stripePaymentHoldId em WeeklyPaymentApproval usam strings em vez de chaves estrangeiras, o que pode causar inconsistencias de dados.

4. Funcionalidades Implementadas

4.1 Landing Page e Paginas Publicas

A plataforma possui uma landing page cinematografica com scroll-linked animations, secoes de servicos ao domicilio (Enfermagem, Fisioterapia, Apoio a Dependentes, Educacao Fisica, Terapia Ocupacional, Auxiliar de Enfermagem), roadmap de teleconsulta (Psicologia, Nutricao, Medicina Geral marcados como "em breve"), pilares de confianca, estatisticas animadas, pricing cards e CTA final. Todas as paginas publicas carregam corretamente: /sobre, /como-funciona, /cuidadores, /familias, /profissionais, /precos, /faq, /ajuda, /contato, /seguranca, /blog, /termos, /privacidade, /cookies, /gdpr.

4.2 Sistema de Registo e Onboarding

- Registo com escolha de perfil (Familia ou Profissional)
- Validacao Zod com mensagens de erro em portugues
- CAPTCHA Cloudflare Turnstile (opcional se nao configurado)
- Aceite obrigatorio de termos (uso, privacidade, mediacao)
- Verificacao de email com token de 24h
- Criacao automatica de perfil (ProfileFamily ou ProfileCaregiver)
- Checagem de duplicados: email unico + telefone unico

4.3 Dashboard Role-Based

O dashboard renderiza de forma diferente para familias e profissionais. Para familias, inclui checklist de onboarding (pagamento, KYC, perfil), hero com estatisticas, contratos recentes, acessos rapidos a cuidados (6 cards), beneficios da plataforma e atividade recente. Para profissionais, inclui hero com ganhos/propostas/ratings, acoes pendentes, propostas pendentes e atividade recente.

4.4 Marketplace de Demandas

Sistema completo onde familias criam demandas de cuidados e opcionalmente pagam por boost de visibilidade. Tres pacotes disponiveis: BASIC (3 EUR/7 dias), PREMIUM (8 EUR/30 dias), URGENTE (15 EUR/3 dias). O algoritmo de ranking prioriza: URGENTE > PREMIUM > BASIC > NONE, seguido por proximidade de expiracao e recencia. Inclui analytics em tempo real (viewCount, proposalCount, conversionRate). Fase 2 completa; Fase 3 (Stripe checkout para boosts e notificacoes) ainda pendente.

4.5 Sistema de Contratos

- Ciclo de vida completo: DRAFT > PENDING_ACCEPTANCE > PENDING_PAYMENT > ACTIVE > COMPLETED
- Aceitacao por ambas as partes com registo de IP e user agent
- Contraproposta suportada (COUNTER_PROPOSED)
- Aprovacao semanal de pagamentos (WeeklyPaymentApproval)
- Sistema de disputas integrado
- Auto-renovacao (planejado, via cron)

4.6 Chat em Tempo Real

Micro-servico separado (Bun, porta 3003) com Socket.IO, acessado via Caddy com suporte WebSocket. Tipos de sala: direta, contrato e suporte. Mensagens suportam texto, imagem, arquivo e mensagens de sistema. Leitura tracking por participante.

4.7 Sistema QR Code

Implementacao completa com 3 endpoints de API (gerar QR, escanear QR, historico), 3 componentes React (QRGenerator com timer de contagem regressiva, QRScanner com camera e fallback manual, QRHistory com tabela e exportacao CSV), rate limiting (5/hora geracao, 10/min escaneamento), trilha de auditoria e formato validado (EYV-{16 chars}).

4.8 Entrevistas por Video

- Integracao com Agora RTC SDK para video-chamadas
- Integracao alternativa com Jitsi Meet para video-conferencia
- Agendamento com questionnaireJson para ambas as partes
- Rotas de API para CRUD de entrevistas

4.9 Internacionalizacao

Suporte a 4 idiomas: portugues (padrao), ingles, italiano e espanhol. Arquivo de traducoes completo (~90KB) com cobertura para navegacao, autenticacao, dashboard, contratos, pagamentos, busca de cuidadores, perfil, configuracoes, notificacoes e erros. Seletor de idioma presente na interface.

5. Sistema de Autenticacao e Autorizacao

5.1 Stack de Autenticacao

Componente	Detalhes
Framework	NextAuth.js 4.24 com estrategia JWT (nao sessoes em DB)
Providers	Google OAuth (condicional) + Credentials (email/senha)
Sessao	JWT custom claims: id, role, status
Hashing	bcrypt com 12 rounds
CSRF	Double-submit cookie com HMAC (Web Crypto API, Edge-compatible)
Middleware	Validacao JWT para rotas /app e /admin, verificacao de role

5.2 Fluxo de Login

O login do administrador (admin@evyra.pt) e tratado de forma especial: auto-cria o admin se nao existir, reseta a senha se a senha padrao for submetida, e corrige o role se estiver errado. Sao tentados 3 caminhos de fallback: Prisma ORM, SQL via db-postgres, e forcar reset. Para utilizadores normais, o fluxo utiliza SQL via db-postgres com comparacao bcrypt e emissao de JWT. O callback JWT forca o role ADMIN para o email admin independentemente do estado do banco de dados.

5.3 RBAC — 5 Niveis de Admin

Role	Capacidades
SUPER_ADMIN	Acesso total, modificar taxas, gerir admins
ADMIN	A maioria das operacoes
SUPPORT	Leitura + suporte ao utilizador

Role	Capacidades
MODERATOR	Moderacao de conteudo apenas
ANALYST	Apenas leitura + analytics

O middleware garante que rotas /app requerem JWT valido e rotas /admin requerem role ADMIN. Utilizadores nao autenticados sao redirecionados para /auth/login com callbackUrl. Nao-admins em /admin sao redirecionados para /app/dashboard. A protecao CSRF utiliza cookie assinado com HMAC (SameSite=Strict, 24h TTL, Secure em producao) comparado com header via comparacao timing-safe. Rotas de webhook e auth sao isentas de CSRF.

6. Arquitetura de Pagamentos

6.1 Principio Fundamental

Evyra e INTERMEDIARIA, nao custodiante. O dinheiro nunca toca nos servidores da Evyra. Isto e fundamental para conformidade PSD2: Evyra = Software Provider (sem licenca necessaria), Stripe = Instituicao de Pagamento (regulada, assegurada ate 100K EUR). Responsabilidade zero da Evyra sobre fundos.

6.2 Tipos de Pagamento

Tipo	Valor	Fluxo	Receita Evyra
Ativacao	35 EUR	Stripe Checkout > Webhook > PENDING>ACTIVE	100%
Taxa de Contrato	5 EUR	Stripe Checkout > Webhook > PENDING_PAYMENT>ACTIVE	100%
Boost Visibilidade	3-15 EUR	Stripe Checkout ou sandbox/mock	100%
Pagamento Servico	Variavel	Stripe Hold > Family Approve > Capture > Connect Transfer	15% comissao

6.3 Fluxo de Escrow

O pagamento de servico utiliza o modelo de escrow do Stripe com capture_method: manual. O fluxo e: (1) Familia autoriza pagamento via Stripe PaymentIntent com captura manual; (2) Stripe coloca fundos em hold; (3) Apos prestacao do servico, familia aprova semanalmente; (4) Evyra captura os fundos; (5) Evyra transfere 85% para o cuidador via Stripe Connect e retm 15% como comissao da plataforma. O sistema de aprovacao semanal permite que a familia approve ou dispute cada semana de servico individualmente.

6.4 Provider Easypay (Portugal)

Para o mercado portugues, a integracao Easypay suporta MB Way, Multibanco (referencia), cartoes de credito/debito e debito direto. A verificacao de webhooks utiliza HMAC-SHA256 com comparacao timing-safe. Validacao de NIF portugues com checksum implementada no cliente.

6.5 Modo Sandbox

Existe um toggle de sandbox no banco (PlatformSettings) que ativa pagamentos mock quando chaves Stripe estão ausentes ou placeholder. Em modo sandbox, pagamentos são auto-completados e criados com provider INTERNAL. Guardas de produção rejeitam fallback mock em produção salvo sandbox explicitamente ativado. Isto é útil para testes mas perigoso se acidentalmente deixado ligado em produção.

7. Sistema KYC (Verificação de Identidade)

A verificação de identidade utiliza o Didit (didit.me), um provedor focado na UE. O fluxo é: (1) createKycSession() cria sessão via API Didit; (2) Utilizador completa verificação na página hospedada do Didit; (3) Webhook Didit com assinatura HMAC-SHA256 verificada; (4) parseVerificationResult() calcula score de confiança (0-100): documento verificado (+50), liveness confirmada (+30), similaridade facial >80% (+20). Auto-aprovação com similaridade >0.85 e liveness; auto-rejeição com <0.5.

Os dados KYC são armazenados no modelo User: kycSessionId, kycSessionToken, kycCompletedAt, kycConfidence, kycBirthDate, kycNationality, kycDocumentIssueDate/ExpiryDate, kycDocumentIssuer, kycData (JSON completo). Existe modo de teste com kycTestMode booleano por utilizador e toggle global. Documentos aceites: passaporte, cartão de cidadão, carta de condução, com idioma português como padrão.

8. Painel Administrativo

O painel administrativo possui arquitetura bem documentada com 5 roles de admin, mais de 10 páginas e dezenas de endpoints de API. Cada ação administrativa é registrada com utilizador, timestamp, IP e detalhes. O sistema suporta impersonação de utilizadores (com log e motivo obrigatório), moderação de conteúdo, gestão de disputas, reembolsos e métricas analíticas.

8.1 Páginas do Admin

Página	Funcionalidades
Dashboard	Estatísticas gerais, crescimento de utilizadores, receita
Utilizadores	Listagem, filtro, suspender, ativar, ver detalhes
Cuidadores	Verificar, rejeitar, destacar, gerir perfis
Contratos	Ver, cancelar, resolver disputas, forçar conclusão
Pagamentos	Aprovar, rejeitar, reembolsar, ver escrow
Demandas	Métricas do marketplace, moderação
Configurações	Taxas, integrações, modo teste, branding (logo/favicon)
Logs	Trilha de auditoria completa
Suporte	Tickets, atribuição, resolução

Pagina	Funcionalidades
Notificacoes	Alertas administrativos por severidade
Moderacao	Fila de moderacao de conteudo
Analytics	Receita, utilizadores, visao geral

9. Estado do Site em Producao

9.1 Saude do Servidor

Metrica	Valor	Status
API Health Check	{"status":"ok","database":"connected"}	OK
Container evyra-app	Up 4 dias	OK
Container evyra-chat	Up 10 dias	OK
Container evyra-postgres	Up 10 dias	OK
CPU/RAM	11GB RAM, 3.9GB usados	OK
Disco	194GB, 61% usado	ATENCAO
Swap	4GB, 1.6GB em uso	ATENCAO

9.2 Erros em Producao

Os logs do container evyra-app mostram erros recorrentes de Prisma com SQL invalido para PostgreSQL. O erro e: *"function date(unknown, unknown) does not exist"* — indicando que o codigo utiliza a funcao DATE() do SQLite (que aceita 2 argumentos para adicionar dias a uma data) que nao existe em PostgreSQL (onde se usa DATE + INTERVAL). Este erro ocorre na rota de analytics admin (user growth), provavelmente migrada do Turso/SQLite sem adaptacao correta para PostgreSQL.

9.3 Problemas de Paginas

Problema	Detalhes	Severidade
Estatisticas zeradas	Homepage mostra 0+ profissionais, 0 paises, 0% satisfacao	ALTA
Contradicao de metricas	Pagina Sobre afirma 50+ profissionais e 100+ familias	ALTA
Zero presenca SEO	Nenhum resultado em motores de busca	ALTA
Telefone placeholder	+351 912 345 678 parece numero template	MEDIA
Paginas duplicadas	/cuidadores e /profissionais com conteudo identico	BAIXA

Problema	Detalhes	Severidade
Dominio inconsistente	Site em evyra.site, OG tags referem evyra.eu	MEDIA

10. Analise de Seguranca — Falhas Criticas

10.1 Falhas Criticas (Vermelho)

- **CREDENCIAIS ADMIN HARDCODED** — A senha admin (EvyraAdmin@2024!) esta em codigo-fonte e no docker-entrypoint.sh. Qualquer pessoa com acesso ao repositorio pode logar como admin. O entrypoint cria o admin com esta senha em cada startup.
- **CSRF COOKIE NAO HTTPONLY** — Por design (JS precisa ler), o cookie CSRF fica exposto a XSS. Se qualquer vulnerabilidade XSS existir, a protecao CSRF e completamente bypassada.
- **SQL INJECTION RISK** — A maior parte da aplicacao usa SQL bruto via db-postgres.ts em vez de Prisma ORM. Se alguma query concatenar input do utilizador em vez de usar parametros, ha risco de SQL injection. Nenhuma auditoria sistematica foi feita nas 90+ rotas de API.
- **ADMIN API KEY COMPARACAO NAO TIMING-SAFE** — validateAdminKey() usa === em vez de crypto.timingSafeEqual(), permitindo ataques de timing para descobrir a chave API.

10.2 Falhas Altas (Laranja)

- **DOIS ARQUIVOS MIDDLEWARE** — middleware.ts (raiz) e src/middleware.ts com responsabilidades sobrepostas. O da raiz so trata sync-turso bypass; o de src trata tudo. Isto e confuso e pode levar a protecoes omitidas.
- **DEV SECRET FALLBACK** — 'dev-secret-for-build' usado como NextAuth secret fallback no middleware e auth config. Se NEXTAUTH_SECRET nao estiver definido, JWTs podem ser forjados.
- **DB PUSH COM --ACCEPT-DATA-LOSS** — O docker-entrypoint.sh executa prisma db push --accept-data-loss como safety net, o que pode silenciosamente descartar colunas em casos limite.
- **SEM RATE LIMITING** — Nenhuma limitacao de taxa visivel nos endpoints de login ou registo, permitindo ataques de forca bruta.
- **CODIGO MORTO NO ENTRYPOINT** — Linhas 177-190 (reset de senha admin) estao apos 'exec node server.js' que substitui o processo, portanto nunca executam.

10.3 Falhas Medias (Amarelo)

- Google OAuth desabilitado silenciosamente se env vars ausentes — nao documentado
- Inconsistencia na taxa de comissao: schema diz 15%, constantes mencionam 10%
- kycTestMode em registros de utilizadores de producao — precisa controle de acesso rigoroso
- Sem reconciliacao automatica Stripe vs DB — risco de transacoes perdidas

- MODO SANDBOX pode ser acidentalmente deixado ligado em producao

10.4 Analise de Headers de Seguranca

O `next.config.ts` implementa headers de seguranca solidos: `X-Frame-Options` (`SAMEORIGIN`), `X-Content-Type-Options` (`nosniff`), `X-XSS-Protection`, `Referrer-Policy` (`strict-origin-when-cross-origin`), `HSTS` com `preload`, `CSP` abrangente (com excecoes para Stripe, Jitsi e CDN), e `Permissions-Policy` restritiva. Isto e positivo e demonstra preocupacao com seguranca.

11. Pontos de Dor e Gargalos

11.1 Arquitetura de Banco de Dados

O sistema utiliza um padrao de adaptador dual (`db-postgres.ts`) que oferece uma interface uniforme para PostgreSQL, Turso e SQLite local. Embora engenhoso, este adaptador introduz complexidade significativa: conversao de placeholders (`?` para `$1`, `$2...`), citacao de identificadores para PostgreSQL, e a possibilidade de bugs sutis na conversao entre dialetos SQL. O erro recorrente da funcao `DATE()` em producao e um exemplo direto deste problema. Alem disso, a maioria da aplicacao usa SQL bruto em vez de Prisma ORM, aumentando o risco de inconsistencias e SQL injection.

11.2 Perfil Monolitico

A pagina de perfil e um componente monolitico de 1544 linhas. A analise documenta que faltam 12 campos criticos para cuidadores (titulo profissional, experiencia, bio, certificacoes, disponibilidade) e 10+ campos para familias (condicoes de saude, medicamentos, restricoes alimentares, mobilidade). O nome da tab "Idoso" e confuso — deveria ser "Familiar". Nao ha suporte para multiplos dependentes. A estimativa de refactoring e de 200-280 horas.

11.3 Migracoes e Deployment

O processo de migracao no Docker entrypoint e excessivamente defensivo: (1) Executa `fix-enums.sql` para converter colunas text para enum; (2) Tenta prisma migrate deploy; (3) Se falhar, faz baseline de TODAS as migracoes e tenta novamente; (4) Executa `prisma db push --accept-data-loss` como safety net. Este padrao sugere historico de problemas com migracoes e e um indicador de divida tecnica. O erro de funcao `DATE()` em producao indica que o codigo ainda contem SQL especifico do SQLite que nao foi adaptado para PostgreSQL.

11.4 Testes

Existem 14 arquivos de teste, mas a qualidade e desigual. O arquivo `boost-complete.test.ts` contem 12 testes que sao todos stubs (`expect(true).toBe(true)`). Nao existem testes de componentes frontend. Os testes QR code sao apenas checklists manuais, nao automatizados. Alguns testes importam de `@/lib/db-turso` mas mockam `@/lib/db-postgres` — potencial mismatch de mocks. A cobertura de validacao (`schemas.test.ts`) nao existe.

11.5 Questoes Legais Bloqueantes

Tres questoes legais exigem opiniao formal antes que as funcionalidades de QR code e historico de visitas possam prosseguir com seguranca: (1) Se a Evyra definir campos do historico de visitas (medicacao, alimentacao, etc.), isto constitui "instrucao de trabalho" sob o codigo laboral portugues, acionando a presuncao de relacao de emprego? (2) Se o historico regista medicacao incorreta e resulta em dano, quem e responsavel? (3) Um QR code gerado pela Evyra constitui "supervisao de trabalho"? Custo estimado: 500-1500 EUR. Prazo: 2-3 semanas.

12. Sugestoes de Melhoria

12.1 Seguranca (Prioridade Critica)

- Mover credenciais admin para variaveis de ambiente com secret management
- Implementar rate limiting em login e registo (express-rate-limit ou similar)
- Usar `crypto.timingSafeEqual()` para comparacao de API keys
- Auditar todas as queries SQL brutas para garantir uso de parametros
- Consolidar middleware em um unico arquivo com logica clara
- Adicionar `httpOnly` ao cookie CSRF ou usar header-based CSRF token
- Remover `dev-secret-for-build` fallback em producao

12.2 Banco de Dados

- Migrar de SQL bruto para Prisma ORM onde possivel — reduz SQL injection risk
- Corrigir funcoes SQL especificas do SQLite para PostgreSQL (`DATE > DATE + INTERVAL`)
- Implementar foreign keys para referencias atualmente armazenadas como strings
- Adicionar job de reconciliacao Stripe vs DB para detectar transacoes perdidas
- Remover `db push --accept-data-loss` do entrypoint — usar apenas `migrate deploy`

12.3 Frontend e UX

- Refatorar pagina de perfil monolitica em componentes role-specific (200-280h estimadas)
- Adicionar campos faltantes: 12 para cuidadores, 10+ para familias
- Suportar multiplos dependentes no perfil de familia
- Sincronizar estatisticas entre homepage e pagina Sobre
- Consolidar paginas duplicadas (`/cuidadores` e `/profissionais`)
- Resolver inconsistencia de dominio (`evyra.site` vs `evyra.eu`)
- Adicionar loading states e error boundaries mais robustos

12.4 SEO e Go-to-Market

- Submeter `sitemap.xml` e `robots.txt` aos motores de busca
- Implementar Server-Side Rendering para conteudo publico (SEO)
- Adicionar Google Analytics ou alternativa privacy-first

- Implementar Open Graph e Twitter Cards mais completos
- Criar estratégia de link building e press outreach
- Adicionar Schema.org markup para serviços de saúde

12.5 Conformidade Legal

- Obter opinião legal formal sobre as 3 questões bloqueantes (500-1500 EUR)
- Finalizar Termos de Serviço com cláusula de intermediário
- Implementar termos de uso do Stripe
- Criar política de reembolso formal
- Garantir cobertura de seguro para falhas de QR/visitas

12.6 Testes

- Substituir stubs em boost-complete.test.ts por testes reais
- Adicionar testes de componentes frontend (React Testing Library)
- Automatizar testes de QR code (atualmente manual)
- Criar schemas.test.ts para validações
- Adicionar testes E2E com Playwright
- Implementar teste de reconciliação Stripe vs DB

12.7 Performance e Escalabilidade

- Implementar cache Redis para queries frequentes (cuidadores disponíveis, etc.)
- Adicionar CDN para assets estáticos (imagens, fontes)
- Implementar paginação cursor-based para grandes listas
- Otimizar queries N+1 no dashboard e listagens
- Adicionar health checks para o micro-serviço de chat
- Monitorar uso de disco (61% atual) — configurar alertas

13. Roadmap — O Que Falta Implementar

Funcionalidade	Status	Esforço Estimado	Prioridade
Aprovação Semanal de Pagamentos	Aprovado, não implementado	3-4 semanas	ALTA
Stripe Checkout para Boosts	Parcialmente documentado	1-2 semanas	ALTA
Sistema de Notificações (email/push)	Não iniciado	2-3 semanas	ALTA

Funcionalidade	Status	Esforço Estimado	Prioridade
Aceitar/Rejeitar Propostas > Contrato	Parcial	1-2 semanas	ALTA
Refatoracao da Pagina de Perfil	Nao iniciado	200-280 horas	ALTA
Validacao Role-Specific na API	Nao implementado	4-6 horas	CRITICA
Chat Familia-Cuidador sobre Propostas	Nao iniciado	1-2 semanas	MEDIA
Rate Limiting por Familia	Nao implementado	1-2 semanas	MEDIA
Auto-renovacao de Contratos (cron)	Nao iniciado	1 semana	MEDIA
Validacoes Geograficas	Nao implementado	1 semana	MEDIA
Testes E2E e Integracao	Parcial	2-3 semanas	MEDIA
Job de Reconciliacao Stripe/DB	Nao iniciado	1 semana	ALTA
Teleconsulta (Psicologia, Nutricao, etc.)	Roadmap	4-8 semanas	BAIXA
Geolocation Tagging	Futuro	2 semanas	BAIXA
Reconhecimento Facial	Futuro	4+ semanas	BAIXA
Modo Offline QR	Futuro	1 semana	BAIXA

O sistema ja possui uma base funcional consideravel com registo, autenticao, marketplace de demandas, contratos, pagamentos Stripe com escrow, KYC via Didit, chat em tempo real e QR code. No entanto, fluxos criticos como a aprovacao semanal de pagamentos e a conversao de propostas em contratos ainda nao estao completos, o que impede a operacao real da plataforma. A falta de validacao role-specific na API e uma vulnerabilidade que pode causar corrupcao de dados e deve ser tratada imediatamente.

14. Conclusao e Recomendacoes Prioritarias

O Evyra e uma plataforma ambiciosa e tecnicamente sofisticada que aborda um problema real e significativo: a coordenacao de cuidados de saude para idosos em Portugal pela diaspورا europela. O stack tecnologico e moderno e bem escolhido, a arquitetura de pagamentos com escrow via Stripe e legalmente robusta, e a cobertura de funcionalidades e abrangente para uma plataforma em fase inicial.

No entanto, a plataforma enfrenta desafios criticos que precisam ser resolvidos antes de qualquer lancamento publico. As credenciais admin hardcoded representam um risco de seguranca inaceitavel. O SQL bruto generalizado sem auditoria sistematica e uma bomba relógio. A ausencia de rate limiting deixa a plataforma vulneravel a ataques de forca bruta. E a inconsistencia de metricas e ausencia total de SEO indicam que a plataforma nao esta pronta para atrair utilizadores reais.

14.1 Acoes Imediatas (1-2 semanas)

- Mover credenciais admin para env vars e trocar senha padrao

- Implementar rate limiting em login e registro
- Auditar queries SQL brutas para SQL injection
- Corrigir funcao DATE() para PostgreSQL em producao
- Adicionar validacao role-specific na API (4-6 horas)

14.2 Acoes de Curto Prazo (1-2 meses)

- Implementar aprovacao semanal de pagamentos
- Completar fluxo Proposta > Contrato
- Implementar sistema de notificacoes
- Obter opiniao legal formal sobre questoes bloqueantes
- Corrigir inconsistencia de metricas e iniciar SEO
- Adicionar job de reconciliacao Stripe vs DB

14.3 Acoes de Medio Prazo (3-6 meses)

- Refatorar pagina de perfil (200-280 horas)
- Migrar SQL bruto para Prisma ORM
- Implementar testes E2E automatizados
- Estrategia de go-to-market e aquisicao de utilizadores
- Considerar CDN e cache Redis para escalabilidade
- Iniciar desenvolvimento de teleconsulta (roadmap)